

Agenda:

1. Review and Questions
2. Transformers

Transformers



Transformers:

Transformers is a relatively new model, It was introduced in 2017 by Google Brain. Used to solve the problem of transduction or transformation of input sequences into output sequences in deep learning applications.

It is an autoencoder structure with self attention.

Main applications are in NLP and computer vision.



Transformers:

How transformers come about:

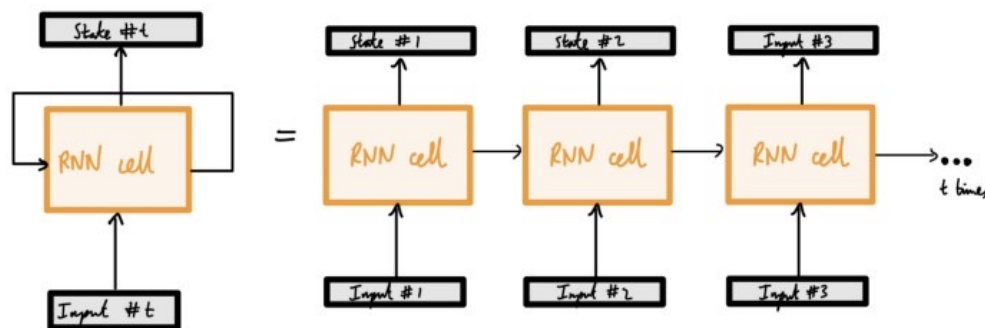
1. RNN
2. Autoencoders
3. Auto-RNN with attention
4. Transformers

Transformers:

RNN:

Are a great use case for NLP as RNNs can remember a lot of information from previous steps. So a great application for RNN is machine translation, as the sequence of words can be remembered (to some degree, they maintained some state which got fed into the next part of the model, along with new input, at each step).

LSTM for translation is a difficult task, as mapping words in a language to another language involves mapping a sequence of arbitrary length to another sequence of arbitrary length.



Transformers:

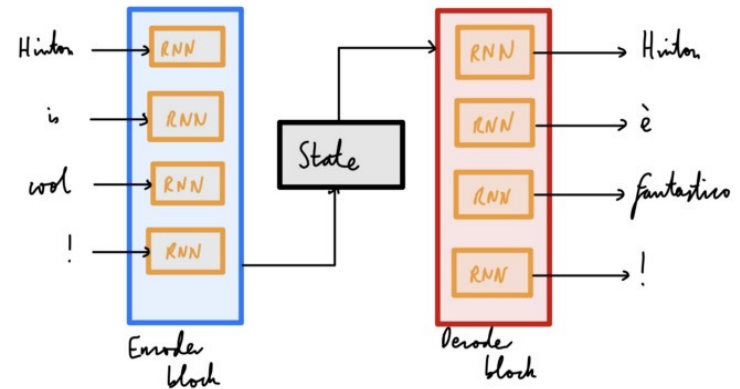
Autoencoder-RNN:

Encoder-decoder architectures are a great fit for this type of problem (translation). The encoder updates the state until the final input, at which the last hidden state is taken into a numerical representation. The decoder is fed this representation (encoded data) and uses it to generate the output sequence.

Since the important information of the original sequence is carried in the context vector, we can see this as the most important part of the architecture, and it may be the cause of "traffic jam" in the system (the information bottleneck caused by the use of only one hidden state was a problem).

The decoder only has access to a very reduced representation of the sequence.

Attention mechanism is the solution.

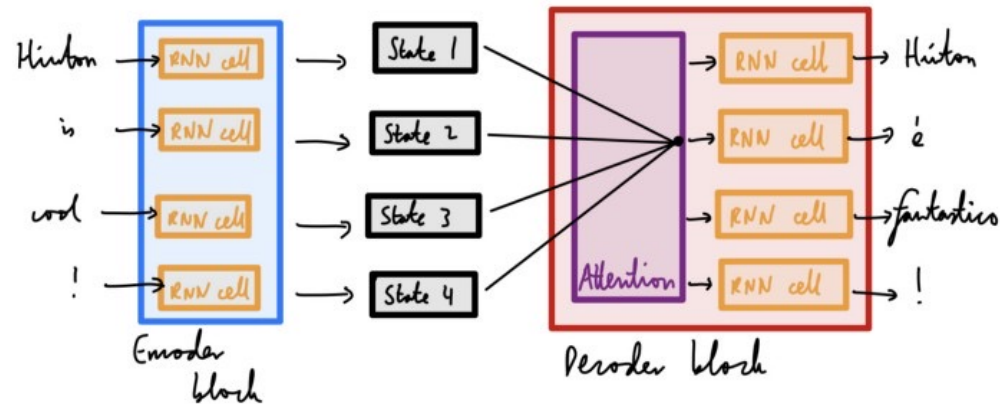


Transformers:

Attention Mechanism:

Originally proposed in 2015 by Minh-Thang Long, and it enables your model to focus on the significant parts of the input sequence as required (rather than the one hidden encoded state).

In a sequence-to-sequence model the encoder passes only the last hidden state as the context to the decoder. When attention is used, the encoder passes not only the last hidden state, but actually all of them.



Transformers:

Attention and self-attention:

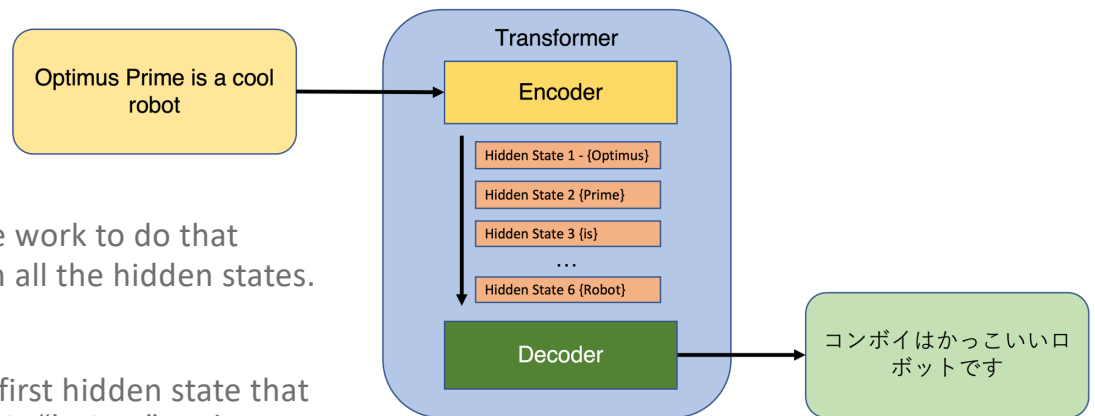
With attention in place, decoder has a lot more work to do than before. The decoder will receive attention from all the hidden states.

In this example, the decoder receives info from the first hidden state that relates to “Optimus” and also the second which is “Prime” and so on...

The encoder now estimates weights based on all of the hidden states from the encoder.

There are a few reasons why this is less than desirable, but one is that it's slow (inputs to be fed in one at a time).

But what about the encoder? (self-attention)



Transformers:

Self-attention:

Looking at the sentence: "Bumblebee plays some catchy music and dances along to it",
We realise that "it" refer to the "music", but this is hard for the machine!

